

KK10 · Relational Database Structure — Digital Vector

When the student council started selling hoodies online, they kept everything in one giant spreadsheet. By week three it was a disaster — Maya's email was spelled three different ways and nobody could update a price without breaking something. This module shows how splitting that chaos into tables joined by keys fixes

What this key knowledge covers

This summary distils everything you need for **KK10 — KK10 · Relational Database Structure — Digital Vector** in Data Analysis. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

One spreadsheet becomes a mess. A relational database keeps it clean.

When the student council started selling hoodies online, they kept everything in one giant spreadsheet. By week three it was a disaster — Maya's email was spelled three different ways and nobody could update a price without breaking something. This module shows how splitting that chaos into tables joined by keys fixes it for good.

What you're accountable for

Key knowledge: **“the structural characteristics of relational databases, including tables, primary keys, foreign keys and queries.”**

Because the dot point uses the word **“including”**, every item after it — tables, primary keys, foreign keys and queries — is directly examinable. You must be able to identify each one, explain what it does, and read a small database to spot them.

A relational database stores data in separate **tables** that are linked together through **keys**. Software that manages one of these is called a **relational database management system** (RDBMS) — Microsoft Access, MySQL and the database behind almost every app you use are all RDBMSs.

The whole idea rests on one principle: **store each fact once, in the table where it belongs, and link to it everywhere else**. Get that, and tables, keys, relationships and queries all click into place. We'll build the intuition with **MerchLine** — the student council's school-merch shop — then drill it with exam-style questions.

The “one big spreadsheet” trap

Here's how MerchLine started — one flat table with a row for every sale. It looks fine at first. Look closer at what repeats.

Maya's name and email are typed out **three times**. The hoodie's price sits in **three** rows. This duplication — storing the same fact in many places — is called data redundancy, and it causes three classic problems examiners love:

- **Update anomaly** — the hoodie price rises to \$48. You must find and change every hoodie row. Miss one and your data now contradicts itself.
- **Insertion anomaly** — you want to add a new product (a tote bag) that nobody has bought yet. But this table only has rows for *sales*, so there's nowhere to put it.
- **Deletion anomaly** — Sam cancels his only order and you delete the row. You've just lost the only record that Sam is even a customer.

Imagine every student carried a full photocopy of the school rulebook. Change one rule and you'd have to recall and reprint **1,200 copies** — and you know some would slip through with the old rule. A relational database is the opposite: **one master rulebook** in the office, and everyone just holds a slip that *points* to it. Change the master once, everyone's up to date.

Tables, records & fields

The fix is to split the data so each table describes exactly **one kind of thing** (one “entity”). MerchLine needs three: the people, the products, and the orders that connect them.

Every table is built from the same two parts:

- A field (a column) is one characteristic — e.g. Email or Price.
 - A record (a row) is one complete entry — all the fields for a single student, product or order.
- Now Maya appears **exactly once**. Her email lives in one cell. Spell it right here, and it's right everywhere it's used. That single move — one fact, one place — is the heart of the whole topic.

A **spreadsheet** also has rows and columns, but it's one loose grid with no enforced links between sheets. A **relational database** enforces structure: defined fields, keys, and relationships the software actively protects. Saying “a database is just a spreadsheet” loses marks.

See the schema in 3D

Here are MerchLine's three tables floating in space. The **Orders** table in the middle is the connector — it holds two foreign keys that point back to the primary keys of Students and Products. **Click an order** to fire the links and watch a single order resolve into a real person and a real product.

Primary keys — the unique handle

A primary key is a field (or combination of fields) whose value **uniquely identifies every record** in a table. No two records can share it, and it can never be empty.

Your school has two students both called **Jack Chen**. Names collide. But each Jack has a different **student ID number** — that's the primary key. The office never looks you up by name; they look you up by the number that can only ever mean *you*.

- **Unique** — no duplicates, ever. The RDBMS will reject a second record with the same key.

- **Not empty (no nulls)** — every record must have one. A record with no key can't be reliably found or linked.
- **Stable** — it shouldn't change. People change surnames and emails; an ID like S-014 stays put, so links never break.

Names repeat (two Jack Chens). Emails change when someone switches providers — and if the email *is* the key, changing it breaks every order linked to it. That's why databases use a dedicated ID field as the key: it's guaranteed unique and never needs to change.

Foreign keys — the link

A foreign key is a field in one table that holds the **primary key value of a record in another table**. It's the actual wire that connects two tables. In MerchLine, the Orders table has two foreign keys:

You hand your coat to the cloakroom and get a ticket stub: **#47**. The stub in your pocket is the foreign key. Coat **#47** hanging on the rack is identified by its primary key. The stub doesn't *contain* your coat — it just points to exactly which coat is yours. Lose the rack and the stub is useless; that's why the link must always point at something real.

The RDBMS enforces a rule: a foreign key must point to a primary key that **actually exists**. You can't create an order for StudentID = S-099 if no such student exists. This protection is called **referential integrity** — it stops “orphan” records that link to nothing.

Relationships & cardinality

The relationship between two tables describes how many records on one side connect to records on the other. This is the **cardinality**, and it's a favourite exam target. Pick one and watch the links form.

Databases can't store a raw many-to-many link directly. One student buys many products; one product is bought by many students. The fix? A **junction table** in the middle (Orders!) that turns one M:N into two clean 1:M relationships. That's literally why the Orders table exists.

Queries — asking questions of the data

A query is a question you ask the database to pull out exactly the records you want. The power of the relational model is that a query can **join tables together** using their keys — combining the person, the product and the order into one readable answer. Pick a question:

You don't walk the whole library re-reading every shelf. You ask the librarian a precise question — “all books by this author, published after 2020” — and they hand back just those. A query is that question; the database is the librarian who knows exactly where everything is because of the keys.

A query doesn't store data — it produces a **view** of data that already lives in the tables. Run it again tomorrow after new orders arrive and you'll get an updated answer. Calling a query “a table” in an exam costs marks.

Sort it: which part is which?

Drag each example into the correct structural component. This is exactly the “identify the feature” task the study design names.

Definitions worth memorising

A database that stores data in separate tables linked together through keys, so each fact is stored once and referenced everywhere else.

A structure that stores data about one kind of thing, organised into records (rows) and fields (columns).

One complete entry in a table — all the fields belonging to a single item, e.g. everything about student S-014.

One characteristic stored for every record in a table, e.g. Email or Price.

A field (or set of fields) whose value uniquely identifies every record in a table. It must be unique and can never be empty.

A field in one table that holds the primary key value of a record in another table, creating the relationship between them.

A request that retrieves and combines records matching set criteria, often joining tables by their keys. It produces a view, not a stored table.

The rule that a foreign key must always point to a primary key that actually exists, preventing orphan records.

Instant-feedback check

Multiple choice, in the style of VCAA Section A. Pick an answer to see why it's right or wrong.

The P-E-L method for written answers

Short-answer database questions reward a tight structure: make a **Point**, ground it in an **Example** from the scenario, then **Link** back to what the question asked. Pick a concept to see it modelled, then try your own.

VCE-style questions · 20 marks

CourtBook is the system that manages bookings for the school's sports courts. It uses three tables: **Members** (everyone allowed to book), **Courts** (court 1, court 2, the gym hall...) and **Bookings** (who booked which court, when). Use this scenario for all questions below.

Attempt each question on paper first. Then reveal the weak answer, the strong answer, and the marking guide — and use the dock to self-mark honestly.

A primary key is an important field in the table. A name is no good because it's too long.

A primary key is a field whose value uniquely identifies each record in a table. A member's name is unsuitable because two different members could share the same name, so it would not be unique and could not reliably identify one record.

- **1 mark** — primary key uniquely identifies each record in a table.
- **1 mark** — a reason based on uniqueness (names can repeat, so not unique / can't reliably identify a record). Accept also: names can change, breaking links.

A foreign key connects the two tables so you know who made the booking. It's in the Bookings table.

A foreign key is a field in the Bookings table that stores the MemberID — the primary key value of a record in the Members table. By matching that value to the primary key in Members, the database links each booking to the one member who made it, without copying the member's

details into the Bookings table.

- **1** — foreign key is a field in Bookings (the ‘many’ side).
- **1** — it holds the primary key value (MemberID) of a record in Members.
- **1** — matching the values links each booking to one member / avoids duplicating member data.

One-screen summary

Store each fact once in its own table, give every record a unique primary key, link tables with foreign keys, and ask questions with queries. Everything in this module is a detail of that sentence.

Common traps & misconceptions

Exam trap

T1 “A primary key can repeat.” It can't — uniqueness is the whole point. Two records with the same primary key would be impossible to tell apart. Fix: state it must be unique and never null.

Exam trap

T2 Putting the foreign key in the wrong table. In a one-to-many relationship the foreign key always lives on the **many** side. StudentID belongs in Orders, not a list of orders crammed into Students. Fix: foreign key goes on the “many” table.

Exam trap

T3 “A database is just a spreadsheet.” A spreadsheet is one loose grid; a relational database has multiple tables with enforced keys and relationships. Fix: name tables, keys and relationships as the difference.

Exam trap

T4 Calling a query a table. A query produces a temporary view of data that lives in tables — it doesn't store its own copy. Fix: say a query retrieves/combines data; it doesn't store it.

Exam trap

T5 Getting the relationship direction backwards. “Many students to one order” is wrong — one order has one buyer. Read it both ways before answering. Fix: test the relationship in both directions.

Exam trap

T6 Defining a key without saying it identifies records. “A primary key is a special field” earns nothing. The mark is in **uniquely identifies each record**. Fix: always include the identification function.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a

judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Define · 2 marks

. Define the term primary key and give one reason a member's name would be unsuitable as the primary key of the Members table. "Define" = state the precise meaning. "Give one reason" = one clear justification.

✗ A weak answer

A primary key is an important field in the table. A name is no good because it's too long. No identification function (the actual definition), and "too long" isn't the real problem. Length isn't why names fail — uniqueness is. Likely 0-1 / 2.

✓ A strong answer

A primary key is a field whose value uniquely identifies each record in a table. A member's name is unsuitable because two different members could share the same name, so it would not be unique and could not reliably identify one record. Definition includes 'uniquely identifies each record' (1 mark) + a correct reason tied to uniqueness (1 mark). Full marks.

How the marks are awarded

- **1 mark** — primary key uniquely identifies each record in a table.
- **1 mark** — a reason based on uniqueness (names can repeat, so not unique / can't reliably identify a record). Accept also: names can change, breaking links.

Q2. Explain · 3 marks

. Explain the role of a foreign key in linking the Bookings table to the Members table. “Explain”
= say what it is AND how it works to achieve the link.

✗ A weak answer

A foreign key connects the two tables so you know who made the booking. It's in the Bookings table. Gestures at the idea but never says a foreign key holds the primary key value of another table. ‘Connects’ without the mechanism caps the mark. ~1 / 3.

✓ A strong answer

A foreign key is a field in the Bookings table that stores the MemberID — the primary key value of a record in the Members table. By matching that value to the primary key in Members, the database links each booking to the one member who made it, without copying the member's details into the Bookings table. Names the field + identifies it as the other table's primary key + explains the matching that creates the link. 3 / 3.

How the marks are awarded

- **1** — foreign key is a field in Bookings (the ‘many’ side).
- **1** — it holds the primary key value (MemberID) of a record in Members.
- **1** — matching the values links each booking to one member / avoids duplicating member data.

Q3. Describe · 5 marks

. Originally CourtBook stored every booking — including the member's full name, email and phone — in a single table. Describe two problems this caused, and explain how restructuring into related tables solves them. “Describe two problems” + “explain how ... solves them.” Marks are split across both halves.

✗ A weak answer

It had lots of repeated data which was messy. Using more tables makes it neater and easier to read. ‘Messy’ and ‘neater’ aren't the named problems. No mention of redundancy, anomalies, or how keys remove them. ~1-2 / 5.

✓ A strong answer

Problem 1 — redundancy: a frequent booker's name, email and phone were stored again in every one of their booking rows, wasting space and inviting inconsistency. **Problem 2 — update anomaly:** if that member changed their email, staff had to edit every booking row, and missing one left the data contradicting itself. **Solution:** splitting into a Members table and a Bookings table stores each member's details once in Members, with each booking holding only a MemberID foreign key that links back. Now a detail is changed in a single place and every linked booking reflects it automatically. Two distinct, correctly named problems (redundancy, update anomaly) each described in the scenario, plus a clear keyed solution. 5 / 5.

How the marks are awarded

- **2 marks** — two correct problems named and described with the scenario (redundancy / update anomaly / insertion / deletion anomaly).
- **1 mark** — restructuring stores each member's details once in a Members table.
- **1 mark** — Bookings links via a MemberID foreign key.
- **1 mark** — a change is then made in one place / integrity maintained.

Q4. Identify · 4 marks

. CourtBook adds a feature: each booking is for exactly one court, and a court can appear in many bookings. Identify the relationship between Courts and Bookings, and justify where the foreign key should be placed. “Identify” the relationship (with direction) + “justify” the key placement with reasoning.

✗ A weak answer

It's a one-to-many relationship. The foreign key goes in a table. Right relationship but no direction stated, and 'goes in a table' gives no justification. ~1-2 / 4.

✓ A strong answer

The relationship is one-to-many: one court can appear in many bookings, but each booking is for exactly one court. The foreign key — CourtID — should be placed in the Bookings table, because the foreign key belongs on the 'many' side. Storing CourtID in each booking lets many bookings each point to one court record, whereas a single Courts record could not list an ever-growing number of booking references in one field. States relationship in both directions, places CourtID in Bookings, and justifies with the 'many-side' rule and why the reverse fails. 4 / 4.

How the marks are awarded

- **1** — identifies one-to-many.
- **1** — states the direction (one court many bookings; each booking one court).
- **1** — foreign key (CourtID) placed in the Bookings table.
- **1** — justification: foreign key sits on the 'many' side / a single field can't hold many references.